

AI Agent Operation Checklists

Purpose: Quick-reference checklists for common operations to ensure compliance with `AI_AGENT_RULES.md`.

Maintenance Rule: When `AI_AGENT_RULES.md` is modified, AI agents **MUST** review and update these checklists to ensure they remain synchronized with the source rules.

Source: These checklists are derived from `Docs/AI_AGENT_RULES.md`. Always refer to the source document for complete details and context.

Checklist 1: File Modification (Code, Scripts, Configuration)

Source: `AI_AGENT_RULES.md` Section "Character Encoding Rules" (lines 177-260)

BEFORE Modifying Any File

- ☐ Check file type: Is it a non-markdown file? (`.py`, `.js`, `.html`, `.css`, `.json`, `.yaml`, `.ps1`, `.bat`, `.sh`, etc.)
 - If YES → Requires 7-bit ASCII (characters 0-127 only)
 - If NO (`.md` file) → May contain UTF-8 but must be PDF-compatible (no emoji)
- ☐ Read encoding rules section in `AI_AGENT_RULES.md` (lines 177-260)
- ☐ Scan file for existing violations:
 - ☐ Check for BOM (UTF-8: `EF BB BF`, UTF-16 LE: `FF FE`, UTF-16 BE: `FE FF`)
 - ☐ Check for non-ASCII characters (use `grep`/search tools)
 - ☐ Check for emoji/Unicode symbols
- ☐ Plan replacements for any non-ASCII characters using common replacements list
- ☐ **Fix existing violations BEFORE making other changes** to the file

DURING Modification

- ☐ Use only 7-bit ASCII characters (0-127 only)
- ☐ Never add emoji, Unicode symbols, or multi-byte characters
- ☐ Replace any existing non-ASCII characters with ASCII equivalents
- ☐ Use common replacements:
 - `×` (multiplication) → `x`
 - `-` (em dash) → `--`
 - `✓` (checkmark) → `[OK]`, `OK`, or `PASS`
 - `✗` (cross) → `[X]`, `FAIL`, or `ERROR`
 - `•` (bullet) → `*` or `-`

- → (arrow) → -> or => or :

AFTER Modification

- ☐ Verify file contains only 7-bit ASCII characters
- ☐ Check for any Unicode/emoji that may have been introduced
- ☐ Verify no BOM present (check first 2-3 bytes)
- ☐ If violations found: Fix immediately before proceeding
- ☐ Test file can be imported/executed (for code files)

Exceptions:

- Markdown files (.md): May contain UTF-8 but must be PDF-compatible
 - Third-party/vendor libraries: Excluded from this rule
-

Checklist 2: Git Commit Operations

Source: AI_AGENT_RULES.md Section "Commit Workflow - Check for Uncommitted Files" (lines 79-116) and "Commit Message Standards" (lines 262+)

BEFORE Committing

- ☐ Check git status: `git status` or `git status --short`
- ☐ Identify which files are part of current change vs. unrelated modifications
- ☐ If uncommitted files exist beyond current change:
 - ☐ Ask user about handling untracked/modified files
 - ☐ Offer options: commit, delete, add to .gitignore, or leave untracked
 - ☐ Wait for user confirmation before proceeding with partial commits
- ☐ Verify commit message follows Angular Conventional Commits format:
 - ☐ Format: `<type>(<scope>): <short summary>`
 - ☐ Types: `feat`, `fix`, `docs`, `style`, `refactor`, `perf`, `test`, `build`, `ci`, `chore`
 - ☐ Blank line between subject and body
 - ☐ Body explains what and why (if needed)
- ☐ If merging to master: Verify using `--no-ff` flag

AFTER Committing

- ☐ Verify commit succeeded (check exit code)
 - ☐ Verify commit message format is correct
 - ☐ Check that all intended files were committed
-

Checklist 3: Terminal Command Execution

Source: AI_AGENT_RULES.md Section "MANDATORY WORKFLOW FOR ALL COMMANDS" (lines 2067-2165)

BEFORE Execution

- ☐ Check relevant project rules: See `.cursor/rules/` for mandatory project rules (Python paths, debugging).
- ☐ Check relevant project rules section:
 - ☐ **Python commands:** Check "Python Script Execution Rules" (lines 1809+)
 - ☐ Use full path: `C:\Users\mbdev\miniforge3\python.exe`
 - ☐ Never use: `python`, `python.exe`, or `py` without full path
 - ☐ **PowerShell commands:** Check "PowerShell Command Syntax Rules" (lines 1934+)
 - ☐ Use `;` for command chaining (NOT `&&`)
 - ☐ Example CORRECT: `cd dir; git init`
 - ☐ Example WRONG: `cd dir && git init`
 - ☐ **Git commands:** Check git workflow rules
- ☐ Verify correct syntax/paths from project documentation
- ☐ Verify prerequisites exist (files, directories, etc.)

AFTER Execution

- ☐ Check exit code IMMEDIATELY:
 - ☐ Non-zero exit code = FAILURE → STOP, read error, fix, retry
 - ☐ Zero exit code = SUCCESS → BUT verify results (exit code 0 doesn't guarantee success for file operations)
- ☐ For file operations (move/copy/delete/rename): Verify both source and destination states
- ☐ Read error messages COMPLETELY (entire error, not just first line)
- ☐ Verify command results (not empty, expected format)
- ☐ If command failed: Address immediately, do NOT proceed with dependent operations
- ☐ Check working directory if path-related errors occur
- ☐ Do NOT ignore errors: Empty output or unexpected results require investigation

Checklist 4: Python Script Execution

Source: AI_AGENT_RULES.md Section "Python Script Execution Rules" (lines 1844+) and `.cursor/rules/project-mandatory.mdc`

Canonical Commands (Single Source of Truth)

- Run script: `C:\Users\mbdev\miniforge3\python.exe script_path.py`
- Run `pytest`: `C:\Users\mbdev\miniforge3\python.exe -m pytest tests/` (or target path)
- Run `py_compile` (after modifying any Python file): `C:\Users\mbdev\miniforge3\envs\cla\python.exe -m py_compile <file_path>`
- Django: `C:\Users\mbdev\miniforge3\python.exe web_app/manage.py <command>`

BEFORE Execution

- ☐ Check "Python Script Execution Rules" section in `AI_AGENT_RULES.md`
- ☐ Identify required Python path: `C:\Users\mbdev\miniforge3\python.exe` (or `envs\cla\python.exe` for `py_compile`)
- ☐ Verify script path is correct (relative or absolute)

DURING Execution

- ☐ Always use `C:\Users\mbdev\miniforge3\python.exe` as Python interpreter
- ☐ Never use `python`, `python.exe`, or `py` without full path
- ☐ Preserve relative paths for script arguments (e.g., `test_code/script.py`)
- ☐ Use full path only for the Python executable itself

AFTER Execution

- ☐ Check exit code (non-zero = failure)
- ☐ Read error messages completely
- ☐ If error: Stop, fix, retry with correct path
- ☐ If success: Verify output is as expected
- ☐ Do NOT proceed with dependent operations if command failed
- ☐ For code changes: Validate Python syntax using `py_compile` before finishing

Checklist 5: Debugging / Bug Investigation

Source: `AI_AGENT_RULES.md` Section "Debugging Rules" and `.cursor/rules/debugging.mdc`

When Investigating a Bug

- ☐ Do NOT infer or guess the root cause. Add temporary diagnostics to confirm the actual cause.
- ☐ Add temporary diagnostics:
 - ☐ Use `logger.warning()` or `logger.error()` so output is visible (e.g. in Docker logs)

- ☐ Use [DIAG] prefix for all diagnostic messages (see Logging Rules)
- ☐ Log actual values (keys, paths, IDs) at the failing call site and at the place that provides the data
- ☐ Use evidence from logs/output to drive the fix. Do not fix based on reasoning alone without confirming.
- ☐ Remove or gate diagnostics when done (remove, comment out, or DEBUG flag). Do not leave temporary diagnostics unless user requests.

For UI / Dashboard Bugs (e.g. "nothing shows", wrong selector)

- ☐ Consider both backend (data, keys, paths, context) and frontend (selectors, data attributes, default state, JS keys)
- ☐ Add diagnostics in both layers if cause is unclear
- ☐ Confirm the mismatch with logs before changing code

Checklist 6: PowerShell Command Execution

Source: AI_AGENT_RULES.md Section "PowerShell Command Syntax Rules" (lines 1969+)

BEFORE Execution

- ☐ Check "PowerShell Command Syntax Rules" section in AI_AGENT_RULES.md
- ☐ Verify command chaining uses ; (NOT &&)
- ☐ Verify syntax is PowerShell-compatible (NOT CMD syntax)

DURING Execution

- ☐ Use ; for command chaining: cmd1; cmd2; cmd3
- ☐ Never use && for command chaining (this is CMD syntax)
- ☐ Use PowerShell-compatible syntax for all commands

AFTER Execution

- ☐ Check exit code
- ☐ Read error messages completely
- ☐ If error mentions "token '&&' is not a valid statement separator": Fix syntax, retry
- ☐ Verify command results

Checklist 7: Git Merge to Master

Source: AI_AGENT_RULES.md Section "WARNING: Merges to Main Branch" (lines 131-139)

BEFORE Merge

- ☐ Check "WARNING: Merges to Main Branch" section in `AI_AGENT_RULES.md`
- ☐ If preparing for release: create draft release notes, announcement, and CHANGELOG on feature branch and commit (Release Workflow Step 2)
- ☐ Verify merge requires user review/approval
- ☐ Generate merge command with `--no-ff` flag: `git merge --no-ff feature/branch-name`
- ☐ Present command to user for review
- ☐ Wait for user to execute merge command

AFTER Merge

- ☐ Verify merge succeeded
 - ☐ Verify merge commit was created (not fast-forward)
 - ☐ Check that all intended changes are in master
-

Checklist 8: Creating Release Tags

Source: `AI_AGENT_RULES.md` Section "WARNING: Creating Tags/Releases" (lines 141-156) and "Release Workflow" (lines 375+)

BEFORE Creating Tag

- ☐ Check "WARNING: Creating Tags/Releases" section in `AI_AGENT_RULES.md`
- ☐ Verify tag creation requires user review/approval
- ☐ Read "Release Workflow" section for complete process (drafts on feature branch, merge, then version bump can review/modify drafts)
- ☐ Update all version references:
 - ☐ `contest_tools/version.py` → Update `__version__`
 - ☐ `README.md` → Update version on line 3
 - ☐ Documentation files (Category A, B, C per `VersionManagement.md`)
 - ☐ In-app help (Category E): Thorough check of `help_dashboard.html`, `help_reports.html`, `help_release_notes.html`, `about.html` per `VersionManagement.md` Section 3
- ☐ VPS update instructions (Step 8): Create `ReleaseNotes/VPS_UPDATE_INSTRUCTIONS_1.0.0-alpha.X.md` with four or five script commands (`cd`, `git fetch --tags`, `git checkout TAG`, `docker compose up --build -d`, optional `verify`); include in version bump or commit after tag; provide user with summary
- ☐ Generate tag command: `git tag -a v1.0.0-alpha.X -m "Release v1.0.0-alpha.X"`
- ☐ Present command to user for review
- ☐ Wait for user to execute tag command

AFTER Creating Tag

- ☐ Verify tag was created successfully
 - ☐ Verify tag points to correct commit
 - ☐ Verify version consistency across all files
-

Checklist 9: Session Initialization

Source: `AI_AGENT_RULES.md` Multiple sections

At Start of Session

- ☐ Read full `AI_AGENT_RULES.md` document
 - ☐ Build mental map of critical sections:
 - ☐ Character Encoding Rules (lines 177-260)
 - ☐ Python Script Execution Rules (lines 1809+)
 - ☐ PowerShell Command Syntax Rules (lines 1934+)
 - ☐ Git Operations Overview (lines 60+)
 - ☐ Command Execution Workflow (lines 2067+)
 - ☐ Check `TECHNICAL_DEBT.md` if it exists (non-blocking but recommended)
 - ☐ Review any uncommitted changes: `git status`
-

Maintenance Checklist: Updating These Checklists

When `AI_AGENT_RULES.md` is modified:

- ☐ Read the modified sections in `AI_AGENT_RULES.md`
 - ☐ Identify which checklists are affected
 - ☐ Update relevant checklists to match new rules
 - ☐ Verify checklist items align with mandatory workflows in source document
 - ☐ Update source line references if document structure changed
 - ☐ Test checklist against recent operations to ensure completeness
-

Notes

- These checklists are derived from `AI_AGENT_RULES.md` and should always be synchronized with the source document.
- If a checklist item conflicts with the source document, the source document takes precedence.
- When in doubt, re-read the relevant section in `AI_AGENT_RULES.md`.
- These checklists are tools to ensure compliance, not replacements for understanding the full rules.